

Relatório de Piloto

LLM Local via Ollama (CPU) no Sistema RAG-ANTT
Simulação do Ambiente de Deploy e Limitações de
Infraestrutura

Agência Nacional de Transportes Terrestres (ANTT)

DeepFeed

Setembro de 2026

Classificação: Relatório técnico de piloto — Projeto RAG-ANTT

O ambiente de desenvolvimento foi configurado para **simular o perfil** de deploy/operação permitido pela ANTT (LLM local via Ollama, inferência em CPU, sem GPU). Este documento registra essa premissa metodológica e os gargalos de infraestrutura **pertinentes a esse perfil**, que limitam o uso de modelos locais mais robustos (acima de $\sim 7B$) na operação prevista.

Sumário

1	Introdução e contexto	3
1.1	Premissa metodológica: desenvolvimento como simulação do deploy . .	3
1.2	Objetivos do piloto	4
1.3	Escopo	4
2	Ambiente de desenvolvimento (simulação do deploy)	4
2.1	Correspondência com o ambiente de operação previsto	5
2.2	Modelos compatíveis com o perfil simulado	6
3	Gargalos pertinentes ao perfil de operação	6
3.1	O que não entra neste diagnóstico	7
3.2	Relação com o ambiente permitido pela Agência	7
3.3	Atendimento simultâneo	7
4	Implicações práticas no perfil de operação	8
5	Como reproduzir o ambiente do piloto	9
6	Conclusão	9

1 Introdução e contexto

O Sistema RAG-ANTT consulta documentos regulatórios da Agência Nacional de Transportes Terrestres (ANTT) por meio de recuperação aumentada (RAG) e geração por modelos de linguagem. No âmbito do ofício SUTEC/ANTT e da arquitetura de referência (`docs/arquitetura_rag_api.tex`), a DeepFeed entrega o núcleo de IA, com previsão de LLM local via Ollama para reduzir dependência de APIs externas e manter dados no perímetro institucional.

1.1 Premissa metodológica: desenvolvimento como simulação do deploy

O piloto **não** foi conduzido como um laboratório de hardware genérico. A estação de desenvolvimento foi deliberadamente operada sob o **mesmo perfil de restrição** esperado no ambiente de deploy/operação da aplicação na Agência (SUTEC/GETIC e cluster Rancher previsto na arquitetura), enquanto não houver acelerador (GPU) e quotas de memória adequadas:

- Inferência **somente em CPU** (sem GPU útil para o runtime do Ollama);
- LLM **local** via Ollama, no perímetro, sem dependência obrigatória de API externa;
- Modelos na faixa praticável sob esse teto (tipicamente 3B–7B quantizados), alinhados ao piso técnico documentado na arquitetura (piloto sem GPU: ordem de 7 cores / 22 GB RAM no cluster-alvo).

A restrição **sem GPU** foi confirmada pela GETIC; a arquitetura de referência formaliza a viabilidade do piloto CPU-only e o dimensionamento mínimo. O notebook usado no desenvolvimento não replica, núcleo a núcleo, a VM de produção, mas **reproduz o perfil operacional decisivo**: CPU-only, memória finita e modelo local Ollama — exatamente as condições sob as quais a aplicação deverá rodar até novo provisionamento.

O objetivo deste relatório é explicitar quais limitações de **infraestrutura do perfil de operação simulado** impedem, no momento, o uso de modelos locais significativamente mais capazes (por exemplo, 13B ou superiores com aceleração por GPU). Não se detalham aqui otimizações de software do pipeline RAG nem incidentes locais de estação que não se transferem para esse perfil de deploy.

1.2 Objetivos do piloto

1. Simular, em desenvolvimento, o perfil de deploy/operação CPU-only previsto para a ANTT;
2. Validar a operação do Ollama nesse perfil (LLM local no fluxo Streamlit do RAG);
3. Confirmar a viabilidade de inferência local com modelos compatíveis com a memória disponível sob CPU-only;
4. Caracterizar os gargalos de infraestrutura **pertinentes ao ambiente de operação simulado**;
5. Fundamentar a necessidade de provisionamento adequado (GPU e/ou mais memória) caso a Agência exija qualidade próxima à de LLMs em nuvem **sem** saída de dados para APIs externas.

1.3 Escopo

Tabela 1: Escopo do piloto versus arquitetura-alvo

Item	Piloto (simulação)	Arquitetura-alvo
Perfil CPU-only / sem GPU	Simulado em desenvolvimento	Previsto até provisionar GPU
Provedor Ollama na interface Streamlit	Sim	Sim (cliente da API)
Modelos 3B e 7B em CPU	Sim	A definir (CPU/GPU no cluster)
Modelos 13B+ com GPU	Não viável no perfil simulado	Condicionado ao provisionamento ANTT
API REST FastAPI	Não	Sim
Deploy Kubernetes (Rancher)	Não (fora do escopo deste piloto)	Sim (ANTT + DeepFeed)

2 Ambiente de desenvolvimento (simulação do deploy)

A tabela abaixo descreve o hardware da estação usada no piloto. Os itens relevantes para a leitura deste documento são aqueles que **espelham o perfil de operação** (CPU-only, RAM finita, Ollama local). Detalhes de estação que não alteram esse perfil — por exemplo, particularidades de instalação no WSL ou episódios pontuais de disco — foram deliberadamente omitidos do diagnóstico de gargalos na Seção 3.

Tabela 2: Perfil de hardware usado para simular o ambiente de operação

Item	Observação
CPU	Intel i5-10300H — suportou inferência quantizada 3B–7B, com latência elevada
RAM	16 GiB totais; tipicamente 8–9 GiB disponíveis sob carga leve (ordem de grandeza compatível com o teto 7B em CPU)
Aceleração	Inferência somente em CPU , alinhada à restrição GETIC (sem GPU no ambiente permitido) e ao piloto CPU-only da arquitetura
Runtime LLM	Ollama (API compatível com OpenAI), modelo de referência qwen2.5:7b

2.1 Correspondência com o ambiente de operação previsto

Tabela 3: O que o desenvolvimento simulou versus o que fica para o cluster

Aspecto	Simulado no desenvolvimento	Deploy-alvo (arquitetura)
Ausência de GPU útil	Sim (CPU-only)	Sem GPU no piloto; GPU sob demanda futura
LLM local (Ollama)	Sim	Pod Ollama no Rancher
Faixa de modelo	3B–7B quantizados	Piso técnico: 7 cores / 22 GB RAM, sem GPU
Dados no perímetro	Sim	Sim (sem saída obrigatória para API externa)
Orquestração K8s / Rancher	Não	Sim
Vários usuários ao mesmo tempo	A API aceita as chamadas em paralelo. A espera surge só na geração local em CPU (padrão: uma por vez). Provedor externo não cria essa fila	O mesmo no cluster: não é limite de infraestrutura do serviço. No Ollama a CPU redige uma geração por vez; no provedor, o paralelismo é do serviço

2.2 Modelos compatíveis com o perfil simulado

Tabela 4: Modelos locais avaliados no piloto

Modelo	Disco	RAM aproximada	Papel no piloto
llama3.2:3b	~2,0 GB	Menor	Baseline rápido; capacidade limitada
qwen2.5:7b	~4,7 GB	~8–10 GiB	Maior modelo local praticável em CPU neste perfil
Modelos 13B+	—	Acima do teto do perfil	Inviáveis sem GPU e mais memória no ambiente de operação

Em suma: sob o perfil de operação simulado (CPU-only, memória finita), a infraestrutura **não comporta** LLMs locais da classe desejável para igualar, de forma estável, a qualidade observada em provedores de nuvem em tarefas normativas densas, sem degradar latência e estabilidade.

3 Gargalos pertinentes ao perfil de operação

Os itens abaixo são exclusivamente aqueles que se transferem do ambiente de desenvolvimento para o **ambiente de deploy/operação simulado** (CPU-only, Ollama local, memória limitada). Incidentes ou atritos de estação que não caracterizam esse perfil foram removidos deste diagnóstico.

Tabela 5: Gargalos de infraestrutura pertinentes ao perfil de operação

ID	Gargalo	Descrição / efeito
I1	Inferência CPU-only	Sem GPU útil no perfil permitido (GETIC / piloto da arquitetura); toda a geração ocorre na CPU, com latência de dezenas de segundos a minutos
I2	RAM insuficiente para modelos maiores	Com memória na ordem de 16 GiB na estação (e piso técnico de 22 GB no cluster-alvo sem GPU), o teto prático fica em modelos ~7B quantizados; 13B+ induzem swap/thrashing ou inviabilidade operacional
I3	Contenção de memória no fluxo RAG	Aplicação (Streamlit/API), embeddings locais e o modelo Ollama competem pela mesma quota de RAM; consultas longas pressionam o teto do perfil
I4	Ausência de GPU no provisionamento ANTT	Enquanto o cluster não oferecer acelerador e quotas adequadas, o perfil de operação permanece CPU-only; o desenvolvimento reproduz essa restrição de propósito

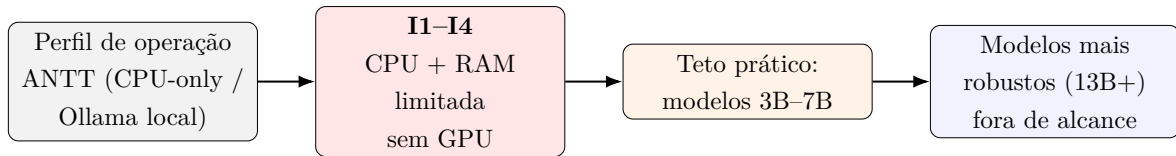


Figura 1: Cadeia causal: o perfil de operação simulado limita o teto do LLM local.

3.1 O que não entra neste diagnóstico

Para manter o documento alinhado à premissa de simulação do deploy, **não** são tratados como gargalos de operação do RAG-ANTT:

- Atritos exclusivos da estação de desenvolvimento (por exemplo, particularidades de WSL, instalação sem privilégios elevados ou episódios pontuais de espaço em disco);
- Otimizações internas do pipeline RAG (packing de contexto, prompts, retrieval) — fora do escopo deste relatório de infraestrutura;
- Ausência, neste piloto, do deploy Kubernetes/Rancher propriamente dito (previsto na arquitetura, não requerido para validar o perfil CPU-only do LLM).

3.2 Relação com o ambiente permitido pela Agência

A decisão de operar LLM **local** atende a requisitos típicos de compliance (dados regulatórios no perímetro, previsibilidade de custo, independência de APIs externas). Contudo, a qualidade da geração local escala com o tamanho do modelo e com a disponibilidade de aceleração por GPU.

No perfil simulado — CPU-only, memória finita, e produção tratada sob a mesma restrição até o provisionamento de GPU no Rancher — a DeepFeed ficou limitada a validar modelos **3B** e **7B**. Isso não é uma escolha arbitrária de produto: é a consequência direta de operar sob as mesmas restrições de infraestrutura esperadas no ambiente de deploy/operação atual da Agência.

Sem alteração desse teto (nós com GPU, quotas de RAM adequadas e runtime com aceleração no cluster), **não é tecnicamente responsável** comprometer o piloto ou a produção com modelos locais maiores, sob risco de instabilidade, latência inaceitável e contenção de memória.

3.3 Atendimento simultâneo

Várias pessoas podem usar a ferramenta ao mesmo tempo. Isso **não** é uma limitação de infraestrutura: a API aceita as chamadas em paralelo, a rede e o índice

não serializam a consulta, e o backend não recusa o segundo usuário. O que impede duas redações locais ao mesmo tempo é a **CPU ocupada pelo modelo**. Uma geração do 7B em CPU usa o processador até terminar; a segunda espera essa geração, não um recurso que falte no cluster.

Ollama em CPU. A variável `RAG_VAGAS_GERACAO_LOCAL` só organiza essa espera. O padrão do piloto é **1**: a segunda pessoa é aceita e fica na vez, no mesmo processo, até a primeira terminar. O código já admite um inteiro maior (teto interno 8), e o processo Ollama pode aceitar mais de uma geração (`OLLAMA_NUM_PARALLEL`). Subir a vaga não remove o gargalo: as gerações extras disputam a mesma CPU e a mesma RAM do modelo. Réplicas ou GPU fariam gerações de verdade em paralelo, e isso sim depende de capacidade no Rancher. Não é o caso deste piloto, e não é o que produz a fila de hoje.

Provedor externo (OpenAI ou DeepSeek). Por esse caminho **não há gargalo de multiusuário**. As chamadas não entram na fila local. Várias pessoas geram ao mesmo tempo, e o processamento paralelo é por conta do serviço, até o limite de taxa e de crédito da conta. A busca nos documentos permanece no perímetro; só o pedido de redação, com o trecho de contexto, sai para a internet. No perfil de produção fechado com a Agência esse caminho permanece desligado (`RAG_LLM_CLOUD_FALLBACK=false`). Desligá-lo é uma escolha de perímetro, não uma falta de capacidade do backend.

4 Implicações práticas no perfil de operação

- Modelos **7B** em CPU são o teto prático do perfil simulado; já exigem cuidado com a quota de memória compartilhada pelo fluxo RAG;
- Modelos **13B ou superiores** são considerados inviáveis nesse perfil sem GPU e sem aumento de RAM no ambiente de operação;
- A primeira carga do modelo 7B no piloto levou da ordem de ~ 1 minuto em pergunta curta; consultas RAG longas aumentam ainda mais o tempo de resposta — latência coerente com inferência CPU-only;
- Enquanto a infraestrutura provisionada permanecer CPU-only, o LLM local deve ser dimensionado para o teto 3B–7B, e expectativas de qualidade devem ser calibradas em relação a esse teto — ou a Agência deve autorizar/provisionar GPU e memória compatíveis com modelos mais robustos;
- Várias pessoas podem consultar ao mesmo tempo. A espera no Ollama local não é falta de infraestrutura: a API aceita as chamadas, e a CPU redige uma geração por vez (`RAG_VAGAS_GERACAO_LOCAL`). Pelo provedor externo esse gargalo não existe; o paralelismo é do serviço.

5 Como reproduzir o ambiente do piloto

```
# 1) Servidor Ollama (CPU) - perfil de operacao simulado
export PATH="$HOME/.local/bin:$PATH"
export LD_LIBRARY_PATH="$HOME/.local/ollama/lib:$LD_LIBRARY_PATH"
ollama serve
ollama list    # llama3.2:3b e qwen2.5:7b

# 2) Aplicacao
cd RAG-ANTT
# Fila local do Ollama. Padrao 1. OpenAI/DeepSeek ignoram esta variavel.
# export RAG_VAGAS_GERACAO_LOCAL=1
streamlit run antt_rag_unified.py

# 3) UI
# Sidebar -> Servico: Local (Ollama) - CPU
# Modelo: qwen2.5:7b
```

6 Conclusão

O piloto demonstrou que é **viável** operar LLM local via Ollama no fluxo Streamlit do RAG-ANTT sob o **perfil de deploy/operação simulado** (CPU-only, sem GPU). Os gargalos determinantes para não adotar modelos locais mais robustos são os que se transferem para esse perfil:

1. Inferência **CPU-only**, alinhada à restrição do ambiente permitido pela ANTT até o provisionamento de GPU;
2. **RAM** insuficiente, nesse perfil, para carregar com estabilidade modelos acima da faixa $\sim 7\text{B}$ quantizados;
3. Contenção de memória entre aplicação, embeddings e o próprio modelo no fluxo RAG;
4. Persistência da ausência de GPU no desenho atual de provisionamento até nova decisão da Agência.

Portanto, a limitação ao uso de LLMs locais mais robustos decorre das **condições de infraestrutura do ambiente de operação que o desenvolvimento visou simular**, e não de uma recusa a priori de modelos maiores nem de atritos locais de estação. A evolução da qualidade do LLM local na ANTT depende, em primeiro

lugar, do **provisionamento** (GPU e/ou mais RAM no cluster Rancher) alinhado aos requisitos de compliance e desempenho da Agência.

O atendimento simultâneo não se confunde com essa limitação de infraestrutura. O backend já recebe várias pessoas ao mesmo tempo. No Ollama, a fila local (padrão: uma vaga) existe porque a geração ocupa a CPU, não porque o serviço ou o cluster recusem a segunda chamada. Em OpenAI ou DeepSeek esse gargalo não aparece: o processamento paralelo fica por conta do provedor. O perfil de produção da Agência segue no modelo local, e por isso a fila permanece.

Documento vivo: atualizar quando mudarem o perfil de infraestrutura ANTT, o hardware usado na simulação do deploy ou o teto de modelo local suportado. Última revisão: setembro de 2026 (premissa: desenvolvimento como simulação do perfil de operação; gargalos restritos a esse perfil; espera multiusuário só na geração local em CPU, inexistente no provedor externo).